

Questão 1

a) O defeito presente na implementação é o fato da função retornar o preço menos 50% do valor total, porém o requisito é dar desconto de 5% caso o cliente seja VIP e não 50%.

b) Chamando, `calcular_preco_final(200, True)`, recebemos a saída 100, porém era para receber 190. Ao rodar o estado interno fica com desconto igual a 100, a falha está em calcular o desconto de forma errada para clientes VIPs.

c)

Com `preco=0`, o defeito de lógica (50% em vez de 5%) não se revela, já que multiplicar 0 por qualquer valor resulta em 0, escondendo o erro. A falha só fica evidente quando o valor base não é nulo.

Reachability: O código `desconto = preco * 0.50` é executado

Infecção: O estado interno é corrompido -> `desconto = 0 * 0.50 = 0`

Propagação: O estado corrompido se propaga -> `retorno = 0 - 0 = 0`

Revelação: FALHA OCULTA — a saída 0 é igual à esperada (50% de desconto em 0 = 0, mas também 5% de desconto em 0 = 0)

d)

Engano: ação humana incorreta (requisito mal interpretado, cálculo errado).

Defeito: consequência do engano (código que aplica 50% em vez de 5%).

Erro: resultado da execução do software quando o defeito é ativado (desconto = 100 para preço 200).

Falha: quando o erro chega ao exterior e provoca saída incorreta (função retorna 100 em vez de 190).

Questão 2

- a) (i) é verificação, porque verifica se o produto foi construído de acordo com a especificação, comparando código e testes com os requisitos. (ii) é validação, porque verifica se o sistema atende às necessidades reais dos usuários, colocando estudantes e servidores para usar uma versão candidata em situações reais de matrícula.
- b) A estratégia da atividade (iii) é normalmente inviável porque o número de combinações possíveis de entradas pode ser muito grande ou até infinito. Mesmo programas simples podem ter muitos campos, valores... Testar tudo exigiria tempo e recursos inviáveis.
- c) Critérios de cobertura ajudam a escolher um conjunto finito de testes ao dividir o domínio de entrada ou o comportamento do programa em subdomínios relevantes. Em vez de testar todas as entradas, escolhe casos que representem esses subdomínios. A rastreabilidade permite relacionar os testes aos requisitos, partes do código ou elementos do critério de cobertura, mostrando o que já foi exercitado. A regra de parada define quando os testes podem ser encerrados, por exemplo, quando todos os requisitos importantes possuem testes associados.
- d) Atingir 100% de um critério de cobertura não prova ausência de defeitos porque o critério mede apenas aquilo que ele foi definido para medir. Por exemplo, 100% de cobertura de comandos não garante que todas as combinações de entradas, caminhos ou requisitos foram testadas.

Questão 3

Implementado em `test_notas.py`

Questão 4

- a) Grafo de fluxo gerado e nomeado: tarifas.pdf
- b) `pytest --cov=tarifa --cov-branch --cov-report=term-missing test_tarifa.py`

c)

O teste fornecido apresenta total cobertura de comandos já que executa os blocos dos dois if's da função, executando o código presente dentro dos mesmos, porém o teste fornecido não cobre todos os desvios, já que não valida os desvios gerados por não executar os blocos dos dois if's.

Os resultados exercitados são $\text{distancia} > 20$ e $\text{urgente} = \text{True}$, enquanto $\text{distancia} \leq 20$ e $\text{urgente} = \text{False}$ permanecem ausentes.

d) Implementado e explicado em `test_tarifa.py`.

e) Um teste que cobre todos os comando passando por todos os if's com condições verdadeiras não cobre os desvios em que as condições dos ifs são falsas. Dessa forma mesmo que um teste tenha 100% de cobertura de comandos ele pode não ter 100% de cobertura de desvios.